



Chapter - 3

Selections

Introduction

- **Conditional Execution:** A program can decide which statements to execute based on a condition.
- **Handling Invalid Input:** If a negative value is entered for radius in a program computing area, it displays an invalid result. To prevent the program from computing the area when the radius is negative, a selection statement can be used.
- **Selection Statements:** Like all high-level programming languages, Java provides selection statements that let you choose actions with alternative courses.

Code Example:

```
if (radius < 0) {  
    System.out.println("Incorrect input");  
}  
else {  
    area = radius * radius * 3.14159;  
    System.out.println("Area is " + area);  
}
```

Boolean Expressions: Selection statements use conditions that are Boolean expressions. A Boolean expression is an expression that evaluates to a Boolean value: true or false.

Boolean Data Type

- **Definition:** The boolean data type declares a variable with the value either true or false.
- **Relational Operators:** Java provides six relational operators (also known as comparison operators) to compare two values, such as whether a radius is greater than 0, equal to 0, or less than 0.

Java Operator	Mathematics Symbol	Name	Example (radius is 5)	Result
<	<	less than	radius < 0	false
<=	"	less than or equal to	radius <= 0	false
>	>	greater than	radius > 0	true
>=	#	greater than or equal to	radius >= 0	true
==	=	equal to	radius == 0	false
!=	\$	not equal to	radius != 0	true

Caution The equality testing operator is two equal signs (==), not a single equal sign (=). The latter symbol (=) is for assignment.

- **Comparison Result:** The result of a comparison is a Boolean value: true or false.



Code Example:

```
double radius = 1;
System.out.println(radius > 0); // This statement displays true
```

- **Boolean Variables:** A variable that holds a Boolean value is known as a Boolean variable. The boolean data type is used to declare Boolean variables.

Code Example:

```
boolean lightsOn = true; // Assigns true to the variable lightsOn
```

- **Boolean Literals:** true and false are literals, just like a number such as 10. They are treated as reserved words and cannot be used as identifiers in the program.

Example: Addition Quiz

Objective: Develop a program to let a first-grader practice addition.

Functionality:

- The program randomly generates two single-digit integers (number1 and number2) and displays a question to the student (e.g., "What is 1 + 7?").
- After the student types the answer, the program displays a message to indicate whether it is true or false.

Generating Random Numbers: For now, the first integer is generated using `System.currentTimeMillis() % 10` and the second using `System.currentTimeMillis() / 7 % 10`.

Program Logic: Lines 5–6 generate the two numbers. Line 14 obtains an answer from the user. The answer is graded in line 18 using the Boolean expression `number1 + number2 == answer`.

```
1 import java.util.Scanner;
2
3 public class AdditionQuiz {
4     public static void main(String[] args) {
5         int number1 = (int)(System.currentTimeMillis() % 10);
6         int number2 = (int)(System.currentTimeMillis() / 7 % 10);
7
8         // Create a Scanner
9         Scanner input = new Scanner(System.in);
10
11         System.out.print(
12             "What is " + number1 + " + " + number2 + "? ";
13
14         int answer = input.nextInt();
15
16         System.out.println(
17             number1 + " + " + number2 + " = " + answer + " is " +
18             (number1 + number2 == answer));
19     }
20 }
```

if Statements

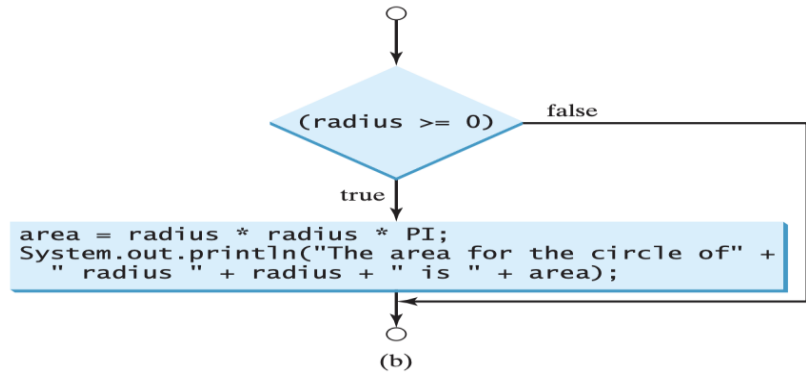
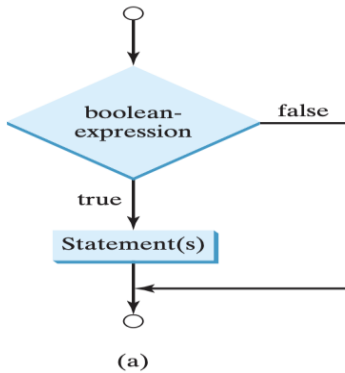
- **Definition:** An if statement is a construct that enables a program to specify alternative paths of execution.
- **Types of Selection Statements:** Java has several types of selection statements:
 - One-way if statements
 - Two-way if-else statements
 - Nested if statements
 - Multi-way if-else statements
 - switch statements
 - Conditional expressions

One-Way if Statements

- **Execution Rule:** A one-way if statement executes an action if and only if the condition is true.

- **Syntax:**

```
if (boolean-expression) {  
    statement(s);  
}
```



An **if** statement executes statements if the **boolean-expression** evaluates to **true**.

- **Flowchart Elements:** A flowchart describes an algorithm or process using boxes and arrows:
 - **Diamond box:** Denotes a Boolean condition.
 - **Rectangle box:** Represents statements.
 - **Arrows:** Represent the flow of control.
- **Execution Logic:** If the boolean-expression evaluates to true, the statements in the block are executed. If the condition is false, the statements inside the block are skipped.

Code Example:

```
if (radius >= 0) {  
    area = radius * radius * PI;  
    System.out.println("The area for the circle of radius " +  
        radius + " is " + area);  
}
```

Parentheses Requirement: The boolean-expression must always be enclosed in parentheses.

Correct vs. Wrong Syntax:

- (a) Wrong:

```
if i > 0 {  
    System.out.println("i is positive");  
}
```
- (b) Correct:

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

Omitting Braces: The block braces can be omitted if they enclose only a single statement. The following styles are equivalent:

```
if (i > 0) {  
    System.out.println("i is positive");  
}
```

(a)

Equivalent

```
if (i > 0)  
    System.out.println("i is positive");
```

(b)

Note: Omitting braces makes the code shorter, but it is prone to errors. It is a common mistake to forget the braces when you go back to modify the code that omits the braces.

Example: Simple if Demo

Objective: Prompts the user to enter an integer.

Logic:

- If the number is a multiple of 5, the program displays HiFive.
- If the number is divisible by 2, it displays HiEven.

```
import java.util.Scanner;

public class SimpleIfDemo {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("Enter an integer: ");
        int number = input.nextInt();

        if (number % 5 == 0)
            System.out.println("HiFive");

        if (number % 2 == 0)
            System.out.println("HiEven");
    }
}
```

Two-Way if-else Statements

- **Definition:** An if-else statement decides the execution path based on whether the condition is true or false.
- **Purpose:** A one-way if statement performs an action if the condition is true and does nothing if it is false. A two-way if-else statement allows you to specify alternative actions for when the condition evaluates to false.

- **Syntax:**

```
if (boolean-expression) {
    statement(s)-for-the-true-case;
}
else {
    statement(s)-for-the-false-case;
```

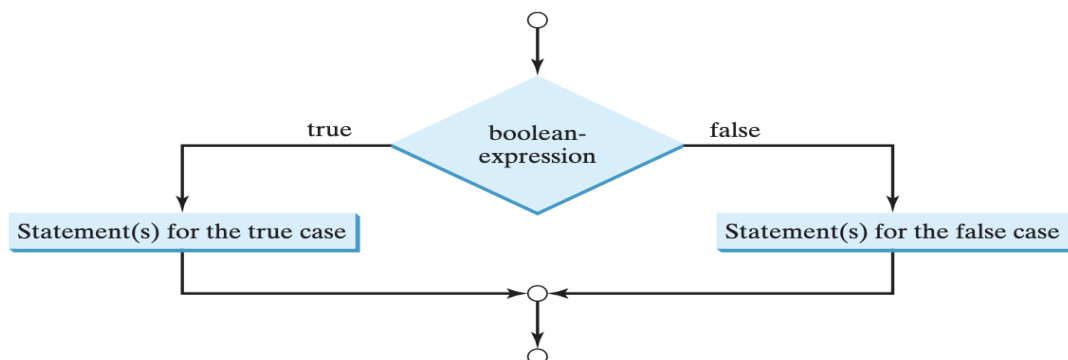


FIGURE 3.2 An **if-else** statement executes statements for the true case if the **Boolean-expression** evaluates to **true**; otherwise, statements for the false case are executed.

- **Execution Logic:** If the boolean-expression evaluates to true, the statement(s) for the true case are executed; otherwise, the statement(s) for the false case are executed.



Code Example (Circle Area Check):

```
if (radius >= 0) {
    area = radius * radius * PI;
    System.out.println("The area for the circle of radius " +
        radius + " is " + area);
}
else {
    System.out.println("Negative input");
}
```

- **Omitting Braces:** Braces can be omitted if there is only a single statement within them. For example, the braces surrounding the else block above can be omitted.

Code Example (Even or Odd Check):

```
if (number % 2 == 0)
    System.out.println(number + " is even.");
else
    System.out.println(number + " is odd.");|
```

Nested if and Multi-Way if-else Statements

- **Definition:** An if statement can be inside another if statement to form a nested if statement.
- **Nesting Depth:** The statement inside an if or if-else statement can be any legal Java statement, including another if or if-else block. There is no technical limit to the depth of the nesting.

Code Example (Nested if):

```
if (i > k) {
    if (j > k)
        System.out.println("i and j are greater than k");
}
else
    System.out.println("i is less than or equal to k");
```

Note: In this example, the if (j > k) statement is nested inside the outer if (i > k) statement.

Multi-Way if-else Alternatives:

- Nested if statements can be used to implement multiple alternatives, such as determining a letter grade based on a numeric score.
- **Execution Process:**
 - The first condition is tested. If true, its block executes.
 - If false, the second condition is tested.
 - This continues sequentially until a condition is met or all conditions prove to be false.
 - A condition is tested only when all of the conditions that come before it evaluate to false.

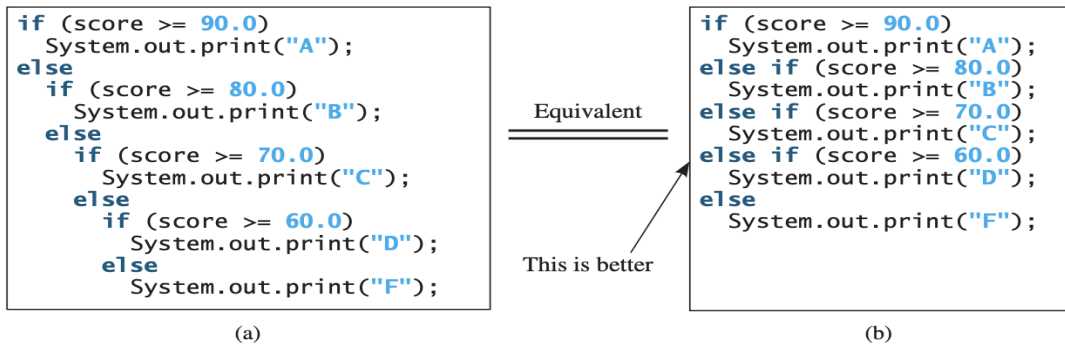


FIGURE 3.3 A preferred format for multiple alternatives is shown in (b) using a multi-way **if-else** statement.

Instead of deeply nesting blocks, using a flattened multi-way if-else style avoids deep indentation and makes the program easier to read.

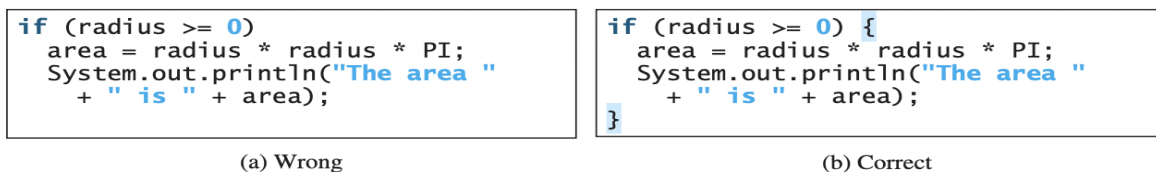
Common Errors and Pitfalls

- **Common Errors:** Forgetting necessary braces, ending an if statement in the wrong place, mistaking `==` for `=`, and dangling else clauses.
- **Common Pitfalls:** Duplicated statements in if-else statements and testing the equality of double values.

Common Error 1: Forgetting Necessary Braces

- Braces can be omitted if the block contains a single statement.
- Forgetting braces when modifying code to add multiple statements groups only the first statement under the if condition, which causes logic bugs.

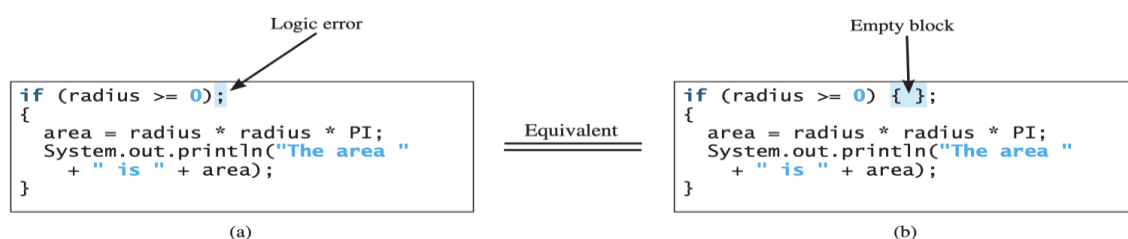
Code Examples:



Common Error 2: Wrong Semicolon at the if Line

- Placing a semicolon directly at the end of an if statement line creates an empty block.
- This is a logic error, not a compilation or runtime error, making it highly difficult to find.
- Using the end-of-line block style instead of the next-line block style can help prevent this mistake.

Code Examples:

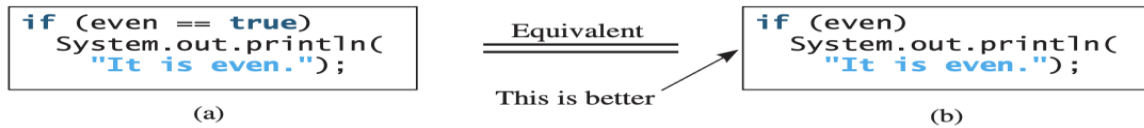


Common Error 3: Redundant Testing of Boolean Values



- Using the equality testing operator (==) to check whether a boolean variable is true or false is redundant.
- Directly testing the variable avoids accidental assignment errors (using = instead of ==), which compile but lead to logic flaws where variables are unintentionally reset.

Code Examples:



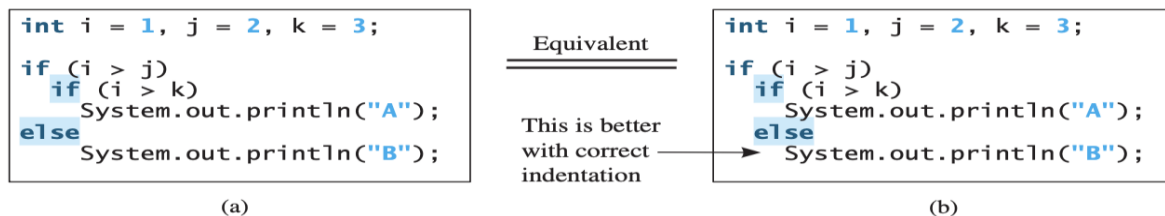
Erroneous Statement (Accidental Assignment):

```
if (even = true) // Assigns true to even; block always executes
    System.out.println("It is even.");
```

Common Error 4: Dangling else Ambiguity

- When multiple if clauses precede a single else clause, indentation can be misleading.
- **Rule:** The else clause always matches the most recent unmatched if clause in the same block.

Code Examples:



(Note: Since $i > j$ is false, nothing is displayed in either case above.)

Forcing the desired match using braces:

```
int i = 1, j = 2, k = 3;
if (i > j) {
    if (i > k)
        System.out.println("A");
}
else
    System.out.println("B"); // Displays "B" because i > j is false
```

Common Error 5: Equality Test of Two Floating-Point Values

- Floating-point numbers have limited precision; calculations can introduce round-off errors, making direct equality tests (==) unreliable.

Code Example (Unreliable Test):

```
double x = 1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1;
System.out.println(x == 0.5); // Displays false because x evaluates to 0.5000000000000001
```

Solution: Check if the absolute difference between the two numbers is less than a tiny threshold value denoted by ϵ (epsilon).

Threshold Standards: Set $\epsilon=10^{-14}$ (1E-14) for double comparisons, and $\epsilon=10^{-7}$ for float comparisons. The `Math.abs(a)` method returns the absolute value.

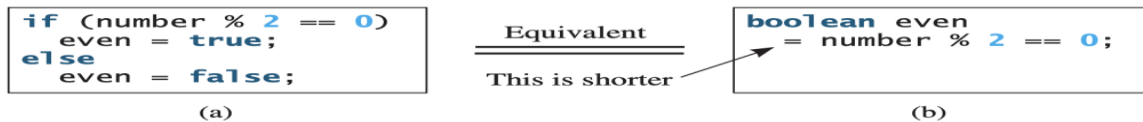
Code Example (Reliable Approximation):



```
final double EPSILON = 1E-14;
double x = 1.0 - 0.1 - 0.1 - 0.1 - 0.1 - 0.1;
if (Math.abs(x - 0.5) < EPSILON)
    System.out.println(x + " is approximately 0.5");
// Displays: 0.5000000000000001 is approximately 0.5
```

Common Pitfall 1: Simplifying Boolean Variable Assignment

- Writing conditional structures to assign values to boolean variables can be simplified by assigning the boolean expression directly.



Common Pitfall 2: Avoiding Duplicate Code in Different Cases

- Writing identical code blocks inside different structural conditions increases maintenance overhead. Moving duplicated statements outside the selection structures fixes this issue.

Code Examples:

(a) Duplicated Structure:

```
if (inState) {
    tuition = 5000;
    System.out.println("The tuition is " + tuition);
}
else {
    tuition = 15000;
    System.out.println("The tuition is " + tuition);
}
```

(b) Refactored / Maintenance-Friendly Structure:

```
if (inState) {
    tuition = 5000;
}
else {
    tuition = 15000;
}
System.out.println("The tuition is " + tuition);
```

Generating Random Numbers

The `Math.random()` Method: Invoking this method returns a random double value d such that $0.0 \leq d < 1.0$ (excluding 1.0).

Better Approach: Using `Math.random()` is a better approach for generating random numbers compared to using `System.currentTimeMillis()`.

Single-Digit Integer Generation: To get a random single-digit integer between 0 and 9, you multiply the result by 10 and cast it to an int:

```
(int)(Math.random() * 10)
```

Case Study: Subtraction Quiz Program

To apply this method, suppose you want to develop a program for a first-grader to practice subtraction. The program randomly generates two single-digit integers, `number1` and `number2`,



and displays a question such as "What is $9 - 2$?". After the student enters the answer, the program displays a message indicating whether it is correct.

To make the quiz practical for a first-grader, the question must always result in a non-negative number. This means the program must guarantee that $\text{number1} \geq \text{number2}$. To handle this dynamically, the program follows these steps:

1. Generate two single-digit integers into `number1` and `number2`.
2. If $\text{number1} < \text{number2}$, swap `number1` with `number2` using a temporary variable.
3. Prompt the student to answer, "What is $\text{number1} - \text{number2}$?"
4. Check the student's answer and display whether it is correct.

The Swapping Mechanism

To swap two variables safely so that no data is overwritten or lost, a temporary container variable (`temp`) is used to first hold the value of `number1`. The value-swapping process executes in three precise steps:

- Line 11: `int temp = number1;` (Stores the original value of `number1` in `temp`)
- Line 12: `number1 = number2;` (Assigns the value of `number2` to `number1`)
- Line 13: `number2 = temp;` (Assigns the saved value from `temp` to `number2`)

Code example SubtractionQuiz.java

```
1 import java.util.Scanner;
2
3 public class SubtractionQuiz {
4     public static void main(String[] args) {
5         // 1. Generate two random single-digit integers
6         int number1 = (int)(Math.random() * 10);
7         int number2 = (int)(Math.random() * 10);
8
9         // 2. If number1 < number2, swap number1 with number2
10        if (number1 < number2) {
11            int temp = number1;
12            number1 = number2;
13            number2 = temp;
14        }
15
16        // 3. Prompt the student to answer "What is number1 - number2?"
17        System.out.print("What is " + number1 + " - " + number2 + "? ");
18        Scanner input = new Scanner(System.in);
19        int answer = input.nextInt();
20
21        // 4. Grade the answer and display the result
22        if (number1 - number2 == answer) {
23            System.out.println("You are correct!");
24        }
25        else {
26            System.out.println("Your answer is wrong.");
27            System.out.println(number1 + " - " + number2 +
28                " should be " + (number1 - number2));
29        }
30    }
31 }
```

Case Study: Computing Body Mass Index



You can use nested if or multi-way if-else statements to write a program that calculates and interprets a person's Body Mass Index (BMI).

Understanding Body Mass Index (BMI)

Body Mass Index (BMI) is a measure of health based on height and weight. It is calculated by taking weight in kilograms and dividing it by the square of height in meters.

For individuals 20 years or older, BMI is interpreted using the following classifications:

BMI	Interpretation
$\text{BMI} < 18.5$	Underweight
$18.5 \leq \text{BMI} < 25.0$	Normal
$25.0 \leq \text{BMI} < 30.0$	Overweight

The program prompts the user to enter their weight in pounds and height in inches, calculates the final BMI using metric conversions, and outputs the corresponding health interpretation category. To perform the necessary metric conversions accurately, the program uses the following fixed measurement standards:

- One pound: 0.45359237 kilograms
- One inch: 0.0254 meters

Defining these values as named constants (`KILOGRAMS_PER_POUND` and `METERS_PER_INCH`) in lines 15–16 makes the program clean, structured, and easy to read. To guarantee full operational accuracy, you should test input values that cover all possible BMI diagnostic categories.



```
1 import java.util.Scanner;
2
3 public class ComputeAndInterpretBMI {
4     public static void main(String[] args) {
5         Scanner input = new Scanner(System.in);
6
7         // Prompt the user to enter weight in pounds
8         System.out.print("Enter weight in pounds: ");
9         double weight = input.nextDouble();
10
11        // Prompt the user to enter height in inches
12        System.out.print("Enter height in inches: ");
13        double height = input.nextDouble();
14
15        final double KILOGRAMS_PER_POUND = 0.45359237; // Constant
16        final double METERS_PER_INCH = 0.0254; // Constant
17
18        // Compute BMI
19        double weightInKilograms = weight * KILOGRAMS_PER_POUND;
20        double heightInMeters = height * METERS_PER_INCH;
21        double bmi = weightInKilograms / (heightInMeters * heightInMeters);
22
23        // Display result
24        System.out.println("BMI is " + bmi);
25        if (bmi < 18.5)
26            System.out.println("Underweight");
27        else if (bmi < 25)
28            System.out.println("Normal");
29        else if (bmi < 30)
30            System.out.println("Overweight");
31        else
32            System.out.println("Obese");
33    }
34 }
```

Case Study: Computing Taxes

You can use multi-way and nested if statements to handle complex business rules, such as calculating personal income tax structures.

Understanding the Tax Calculation Logic

The United States federal personal income tax is calculated based on two inputs: filing status and taxable income.

1. **Filing Statuses:** There are four standard filing options, mapped to numerical codes in the program:
 - 0: Single Filers
 - 1: Married Filing Jointly or Qualified Widow(er)
 - 2: Married Filing Separately
 - 3: Head of Household
2. **Marginal Tax Brackets:** Tax rates vary annually. The rates are marginal, meaning income is not taxed entirely at a single rate. Instead, it is partitioned into separate brackets, and each segment is taxed at its corresponding percentage rate.



Marginal Tax Rate	Single	Married Filing Jointly or Qualifying Widow(er)	Married Filing Separately	Head of Household
10%	\$0 – \$8,350	\$0 – \$16,700	\$0 – \$8,350	\$0 – \$11,950
15%	\$8,351 – \$33,950	\$16,701 – \$67,900	\$8,351 – \$33,950	\$11,951 – \$45,500
25%	\$33,951 – \$82,250	\$67,901 – \$137,050	\$33,951 – \$68,525	\$45,501 – \$117,450
28%	\$82,251 – \$171,550	\$137,051 – \$208,850	\$68,526 – \$104,425	\$117,451 – \$190,200
33%	\$171,551 – \$372,950	\$208,851 – \$372,950	\$104,426 – \$186,475	\$190,201 – \$372,950
35%	\$372,951+	\$372,951+	\$186,476+	\$372,951+

Example calculation: For a single filer with \$400,000 of taxable income, the tax computation breaks down as follows:

- First \$8,350 is taxed at 10%
- The next (\$33,950 – \$8,350) is taxed at 15%
- The next (\$82,250 – \$33,950) is taxed at 25%
- The next (\$171,550 – \$82,250) is taxed at 28%
- The next (\$372,950 – \$171,550) is taxed at 33%
- The remaining (\$400,000 – \$372,950) is taxed at 35%

The primary decision structure splits the logic based on the user's filing status using a multi-way if-else statement. Within each filing status branch, another nested multi-way if-else structure handles the progressive calculation matching the corresponding income bracket tiers.

```

if (status == 0) {
    // Compute tax for single filers
}
else if (status == 1) {
    // Compute tax for married filing jointly or qualifying widow(er)
}
else if (status == 2) {
    // Compute tax for married filing separately
}
else if (status == 3) {
    // Compute tax for head of household
}
else {
    // Display wrong status
}

```

- **Variable Initialization Rule:** The tax variable must be explicitly initialized with a starting value (`double tax = 0;`). Because all subsequent assignments to tax take place inside



conditional code blocks, the compiler cannot guarantee that any assignment line will execute. Leaving it uninitialized would cause a compile error.

- **Program Termination via `System.exit()`:** Invoking the utility method `System.exit(status)` immediately shuts down the program. Passing a status argument of 0 signals a clean, normal exit, whereas a non-zero argument (such as `System.exit(1)`) indicates an abnormal termination.
- **Testing Requirements:** To thoroughly test this program, input validation must cover all 4 statuses. Each status contains 6 progressive tax brackets, making a total of 24 test conditions.

Tip: *Incremental Development and Testing* For all programs, you should write a small amount of code and test it before moving on to add more code. This approach makes testing easier, because errors are likely contained in the new code you just added.

LISTING 3.5 ComputeTax.java



```
1 import java.util.Scanner;
2
3 public class ComputeTax {
4     public static void main(String[] args) {
5         // Create a Scanner
6         Scanner input = new Scanner(System.in);
7
8         // Prompt the user to enter filing status
9         System.out.print("(0-single filer, 1-married jointly or " +
10             "qualifying widow(er), 2-married separately, 3-head of " +
11             "household)\nEnter the filing status: ");
12         int status = input.nextInt();
13
14         // Prompt the user to enter taxable income
15         System.out.print("Enter the taxable income: ");
16         double income = input.nextDouble();
17
18         // Compute tax
19         double tax = 0;
20
21         if (status == 0) { // Compute tax for single filers
22             if (income <= 8350)
23                 tax = income * 0.10;
24             else if (income <= 33950)
25                 tax = 8350 * 0.10 + (income - 8350) * 0.15;
26             else if (income <= 82250)
27                 tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
28                     (income - 33950) * 0.25;
29             else if (income <= 171550)
30                 tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
31                     (82250 - 33950) * 0.25 + (income - 82250) * 0.28;
32             else if (income <= 372950)
33                 tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
34                     (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 +
35                     (income - 171550) * 0.33;
36             else
37                 tax = 8350 * 0.10 + (33950 - 8350) * 0.15 +
38                     (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 +
39                     (372950 - 171550) * 0.33 + (income - 372950) * 0.35;
40         }
41         else if (status == 1) { // Left as an exercise
42             // Compute tax for married file jointly or qualifying widow(er)
43         }
44         else if (status == 2) { // Compute tax for married separately
45             // Left as an exercise
46         }
47         else if (status == 3) { // Compute tax for head of household
48             // Left as an exercise
49         }
50         else {
51             System.out.println("Error: invalid status");
52             System.exit(1);
53         }
54
55         // Display the result
56         System.out.println("Tax is " + (int)(tax * 100) / 100.0);
57     }
58 }
```

Logical Operators

Purpose: Logical operators, also known as Boolean operators, operate on Boolean values to create a new compound Boolean value. They allow you to combine multiple conditions to determine whether a statement should execute.

Operator	Name	Description
----------	------	-------------



!	not	logical negation
&&	and	logical conjunction
	or	logical disjunction
^	exclusive or	logical exclusion

Note on XOR (^): The expression $p1 \wedge p2$ produces the exact same result as the inequality comparison $p1 \neq p2$.

TABLE 3.4 Truth Table for Operator !

p	!p	Example (assume age = 24, weight = 140)
true	false	!(age > 18) is false, because (age > 18) is true.
false	true	!(weight == 150) is true, because (weight == 150) is false.

TABLE 3.5 Truth Table for Operator &&

p ₁	p ₂	p ₁ && p ₂	Example (assume age = 24, weight = 140)
false	false	false	
false	true	false	(age > 28) && (weight <= 140) is true, because (age > 28) is false.
true	false	false	
true	true	true	(age > 18) && (weight >= 140) is true, because (age > 18) and (weight >= 140) are both true.

TABLE 3.6 Truth Table for Operator ||

p ₁	p ₂	p ₁ p ₂	Example (assume age = 24, weight = 140)
false	false	false	(age > 34) (weight >= 150) is false, because (age > 34) and (weight >= 150) are both false.
false	true	true	
true	false	true	(age > 18) (weight < 140) is true, because (age > 18) is true.
true	true	true	

TABLE 3.7 Truth Table for Operator ^

p ₁	p ₂	p ₁ ^ p ₂	Example (assume age = 24, weight = 140)
false	false	false	(age > 34) ^ (weight > 140) is false, because (age > 34) and (weight > 140) are both false.
false	true	true	(age > 34) ^ (weight >= 140) is true, because (age > 34) is false but (weight >= 140) is true.
true	false	true	
true	true	false	

Short-Circuit (Lazy) Evaluation



Java optimizes performance by evaluating compound Boolean expressions from left to right and stopping as soon as the final outcome is guaranteed. Because of this, `&&` and `||` are known as short-circuit or lazy operators.

- **For `p1 && p2`:** Java evaluates `p1` first. If `p1` is false, the entire expression is guaranteed to be false, so `p2` is not evaluated.
- **For `p1 || p2`:** Java evaluates `p1` first. If `p1` is true, the entire expression is guaranteed to be true, so `p2` is not evaluated.

(Note: Java also provides unconditional AND (`&`) and OR (`|`) operators that always evaluate both operands, which are covered in advanced supplements.)

Structural Limitations and Syntax Rules

Caution In mathematics, the chained inequality expression `1 <= numberOfDaysInAMonth <= 31` is perfectly valid. However, this is incorrect in Java.

The structural evaluation happens step-by-step: `1 <= numberOfDaysInAMonth` evaluates first and results in a boolean value (true or false). Java cannot compare a boolean value to a numeric literal like 31. You must connect two distinct relational comparisons using a logical operator instead:

```
(1 <= numberOfDaysInAMonth) && (numberOfDaysInAMonth <= 31)
```

De Morgan's Law

Named after the logician Augustus De Morgan, this rule is used to simplify complex, negated Boolean expressions:

- `!(condition1 && condition2)` is logically equivalent to `!condition1 || !condition2`
- `!(condition1 || condition2)` is logically equivalent to `!condition1 && !condition2`

Simplification Examples:

- `!(number % 2 == 0 && number % 3 == 0)` simplifies directly to `(number % 2 != 0 || number % 3 != 0)`
- `!(number == 2 || number == 3)` is much cleaner when written as `number != 2 && number != 3`

LISTING 3.6 TestBooleanOperators.java

This program prompts a user for an integer and evaluates it against multiple conditions to see how the numbers interact with `&&`, `||`, and `^`.



```
1 import java.util.Scanner;
2
3 public class TestBooleanOperators {
4     public static void main(String[] args) {
5         // Create a Scanner
6         Scanner input = new Scanner(System.in);
7
8         // Receive an input
9         System.out.print("Enter an integer: ");
10        int number = input.nextInt();
11
12        // Line 12: Checks if the number is divisible by BOTH 2 and 3
13        if (number % 2 == 0 && number % 3 == 0)
14            System.out.println(number + " is divisible by 2 and 3.");
15
16        // Line 15: Checks if the number is divisible by 2 OR by 3 (or both)
17        if (number % 2 == 0 || number % 3 == 0)
18            System.out.println(number + " is divisible by 2 or 3.");
19
20        // Line 18: Checks if the number is divisible by 2 or 3, BUT NOT BOTH
21        if (number % 2 == 0 ^ number % 3 == 0)
22            System.out.println(number + " is divisible by 2 or 3, but not both.");
23    }
24 }
```

Case Study: Determining Leap Year

You can use compound Boolean expressions with logical operators to implement complex calendar rules, such as determining whether a given year is a leap year.

Mathematical Logic for Leap Years

A year is classified as a leap year if it meets specific astronomical and division-based criteria:

1. It must be divisible by 4 but not divisible by 100, OR
2. It must be divisible by 400.

Constructing the Boolean Expression

This multi-layered rule can be broken down step-by-step into individual operational assignments:

```
// Step 1: A leap year must be divisible by 4
boolean isLeapYear = (year % 4 == 0);
```

```
// Step 2: It must also not be divisible by 100
isLeapYear = isLeapYear && (year % 100 != 0);
```

```
// Step 3: Or, it can simply be divisible by 400
isLeapYear = isLeapYear || (year % 400 == 0);
```

Optimized Compound Expression

Instead of using multiple lines of code and reassignment statements, you can combine these conditions into a single, clean compound Boolean expression. Operator precedence ensures the && operations group together tightly before the || operation is evaluated:

```
boolean isLeapYear = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
```



```
1 import java.util.Scanner;
2
3 public class LeapYear {
4     public static void main(String[] args) {
5         // Create a Scanner
6         Scanner input = new Scanner(System.in);
7         System.out.print("Enter a year: ");
8         int year = input.nextInt();
9
10        // Check if the year is a leap year using a compound expression
11        boolean isLeapYear =
12            (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
13
14        // Display the result
15        System.out.println(year + " is a leap year? " + isLeapYear);
16    }
17 }
```

Case Study: Lottery

The lottery program demonstrates how to solve a multi-tiered decision problem by combining random number generation, digit extraction arithmetic, and conditional selection structures with Boolean operators.

Game Rules and Award Criteria

The program generates a secret, two-digit target number and prompts the user to guess it. Winners are determined based on three precise validation tiers:

1. **Exact Match (\$10,000 Award):** The user's input matches the lottery number in the exact same order.
2. **All Digits Match (\$3,000 Award):** Every digit in the user's input matches the digits in the lottery number, but out of order (i.e., a reversed match).
3. **One Digit Matches (\$1,000 Award):** At least one digit from the user's input matches any single digit within the secret lottery number.
- **Two-Digit Padded Assumption:** Digits within a two-digit number may include 0. Numbers below 10 are treated as if they have a leading zero (e.g., 8 behaves as 08, and 0 behaves as 00).

Digit Extraction and Program Architecture

To evaluate the guess against the lottery number, the program breaks both two-digit integers down into individual digits using basic integer arithmetic:

- **Extracting the tens digit (/ 10):** Performing integer division by 10 removes the last digit, leaving only the first digit (e.g., $45/10=4$).
- **Extracting the ones digit (% 10):** The remainder operator returns the last digit of the number (e.g., $45\%10=5$).

Conditional Evaluation Hierarchy

The conditional matching logic must check requirements from highest prize to lowest prize. If the check order is scrambled, lower-tier rules might trigger falsely on a perfect match:

1. **guess == lottery:** Checks for an exact match first.



2. `guessDigit2 == lotteryDigit1 && guessDigit1 == lotteryDigit2`: Checks if the digits match completely in reverse order.
3. `guessDigit1 == lotteryDigit1 || guessDigit1 == lotteryDigit2 || ...`: Uses the conditional OR operator (`||`) to check if any single digit from the guess matches any digit in the target number.
4. `else`: Acts as a catch-all block when no match occurs.

LISTING 3.8 Lottery.java

```
1 import java.util.Scanner;
2
3 public class Lottery {
4     public static void main(String[] args) {
5         // Generate a secret lottery number between 0 and 99
6         int lottery = (int)(Math.random() * 100);
7
8         // Prompt the user to enter a two-digit guess
9         Scanner input = new Scanner(System.in);
10        System.out.print("Enter your lottery pick (two digits): ");
11        int guess = input.nextInt();
12
13        // Get individual digits from the secret lottery number
14        int lotteryDigit1 = lottery / 10;
15        int lotteryDigit2 = lottery % 10;
16
17        // Get individual digits from the user's guess
18        int guessDigit1 = guess / 10;
19        int guessDigit2 = guess % 10;
20
21        System.out.println("The lottery number is " + lottery);
22
23        // Check the guess using a structured cascading tier
24        if (guess == lottery)
25            System.out.println("Exact match: you win $10,000");
26        else if (guessDigit2 == lotteryDigit1 && guessDigit1 == lotteryDigit2)
27            System.out.println("Match all digits: you win $3,000");
28        else if (guessDigit1 == lotteryDigit1
29                || guessDigit1 == lotteryDigit2
30                || guessDigit2 == lotteryDigit1
31                || guessDigit2 == lotteryDigit2)
32            System.out.println("Match one digit: you win $1,000");
33        else
34            System.out.println("Sorry, no match");
35    }
36 }
```

Switch Statements

Overusing nested if statements can make code difficult to read. Java provides a switch statement as a clean alternative to simplify coding when handling multiple fixed conditions. For example, the multi-way filing status checks in the tax program can be refactored using a switch statement block:



```
switch (status) {
    case 0: compute tax for single filers;
            break;
    case 1: compute tax for married jointly or qualifying widow(er);
            break;
    case 2: compute tax for married filing separately;
            break;
    case 3: compute tax for head of household;
            break;
    default: System.out.println("Error: invalid status");
            System.exit(1);
}
```

Full Syntax and Structural Rules

The general structure of a switch statement is defined as follows:

```
switch (switch-expression) {
    case value1: statement(s)1;
                break;
    case value2: statement(s)2;
                break;
    ...
    case valueN: statement(s)N;
                break;
    default: statement(s)-for-default;
}
```

The syntax follows these specific constraints:

- **Allowed Data Types:** The switch-expression must always be enclosed in parentheses and must evaluate to a value of type char, byte, short, int, or String.
- **Constant Expressions:** Each target value inside the case labels must match the data type of the switch-expression. These values must be literal constants or constant expressions; they cannot contain variables (e.g., $1 + x$ is invalid).
- **Sequential Execution:** When an expression matches a case value, the program executes statements sequentially from that point forward until it hits a break statement or reaches the closing brace of the switch block.
- **The default Case:** This optional block handles fallback actions when none of the defined explicit case values match the evaluated expression.

Fall-Through Behavior and Fault Mitigation

Caution The break keyword is completely optional, but omitting it causes the program to fall through to subsequent cases. Once a case matches, all following case statements run until a break or the end of the switch block is reached.

Intentional Fall-Through Example

Sometimes, you can intentionally leverage fall-through behavior to group multiple matching values under a single execution path. The code below maps days 1–5 to weekdays and days 0 and 6 to weekends:

```
switch (day) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5: System.out.println("Weekday"); break;
    case 0:
    case 6: System.out.println("Weekend");
}
```

Tip: To avoid accidental logic errors and improve code maintainability, it is a good engineering practice to add an explicit comment in the source code whenever a break statement is purposely omitted.

LISTING 3.9 ChineseZodiac.java

The Chinese Zodiac maps animal signs to individuals over a twelve-year cycle. You can calculate the exact sign using the remainder operator: $\text{year} \% 12$. For example, since $1900 \% 12$ evaluates to 4, the year 1900 is mapped to the year of the rat.

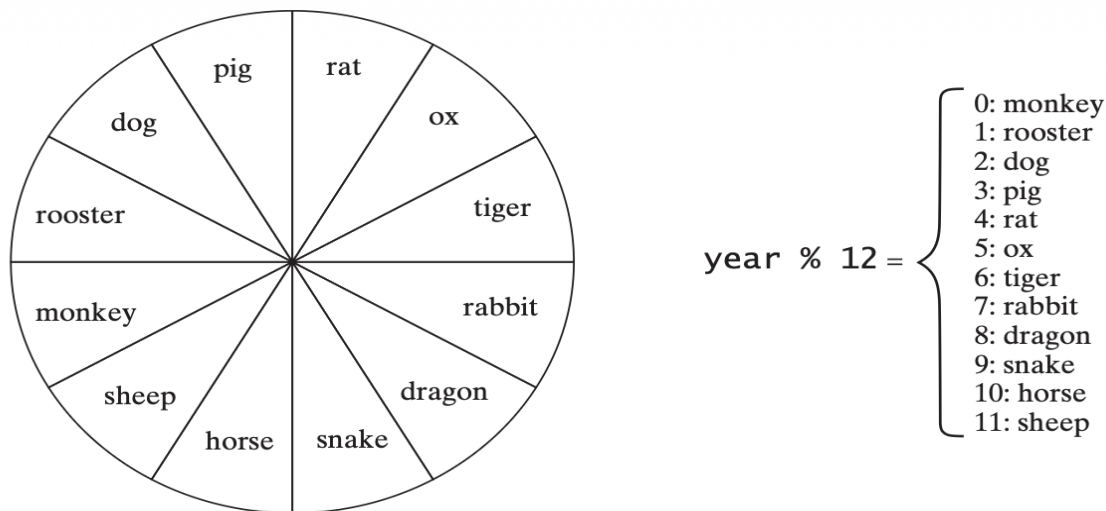


FIGURE 3.6 The Chinese Zodiac is based on a twelve-year cycle.

```
import java.util.Scanner;

public class ChineseZodiac {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        System.out.print("Enter a year: ");
        int year = input.nextInt();

        // Check cycle remainder and print corresponding animal sign
        switch (year % 12) {
            case 0: System.out.println("monkey"); break;
            case 1: System.out.println("rooster"); break;
            case 2: System.out.println("dog"); break;
            case 3: System.out.println("pig"); break;
            case 4: System.out.println("rat"); break;
            case 5: System.out.println("ox"); break;
            case 6: System.out.println("tiger"); break;
            case 7: System.out.println("rabbit"); break;
            case 8: System.out.println("dragon"); break;
            case 9: System.out.println("snake"); break;
            case 10: System.out.println("horse"); break;
            case 11: System.out.println("sheep"); break;
        }
    }
}
```

Conditional Expressions

A conditional expression allows you to evaluate and assign a value to a variable based on a specific condition. It serves as a short, highly concise alternative to standard if-else selection blocks.

For example, consider an execution path that assigns a value to y based on x :



```
// Using standard if-else
if (x > 0)
    y = 1;
else
    y = -1;

// Using an equivalent conditional expression
y = (x > 0) ? 1 : -1;
```

Syntax and Operational Rules

Conditional expressions use a unique inline style with no explicit if keyword. The general syntax is structured as follows:

```
boolean-expression ? expression1 : expression2;
```

- **True Case:** If the boolean-expression resolves to true, the entire statement evaluates to the value of expression1.
- **False Case:** If the boolean-expression resolves to false, the entire statement evaluates to the value of expression2.

Practical Examples

1. Finding the Maximum Value

To find the larger of two variables (num1 and num2) and assign it to max:

```
max = (num1 > num2) ? num1 : num2;
```

2. Inline Printing (Even or Odd Check)

You can embed conditional expressions directly inside system output methods to clean up print logic:

```
System.out.println((num % 2 == 0) ? "num is even" : "num is odd");
```

Note: The Ternary Operator The ? and : symbols work together to form the conditional operator. It is also commonly referred to as a ternary operator because it requires exactly three operational operands to function. It is the only ternary operator available in the Java programming language.

Operator Precedence and Associativity

Operator precedence and associativity determine the exact order in which operators are evaluated within an expression. For complex expressions containing multiple operational types—such as arithmetic, relational, and logical operators—these structural rules prevent ambiguity during runtime execution.

Consider the complex expression:

```
3 + 4 * 4 > 5 * (4 + 3) - 1 && (4 - 3 > 5)
```

To find its value, Java applies explicit rules of encapsulation, precedence tiers, and grouping directions.

The Fundamental Rules of Ordering

1. **Parentheses Rules:** Expressions enclosed within parentheses () are always isolated and evaluated first. If parentheses are nested, evaluation begins inside the innermost set and works outward.
2. **Precedence Rule:** When evaluating expressions without parentheses, operators are applied based on a strict hierarchy. Highly prioritized operators are executed before lower-priority ones.



3. **Associativity Rule:** If two adjacent operators have the exact same precedence level, their associativity rule determines whether they are evaluated from left-to-right or right-to-left.

TABLE 3.8 Operator Precedence Chart

Precedence	Operator
↓	var++ and var-- (Postfix)
	+, - (Unary plus and minus), ++var and --var (Prefix)
	(type) (Casting)
	! (Not)
	*, /, % (Multiplication, division, and remainder)
	+, - (Binary addition and subtraction)
	<, <=, >, >= (Relational)
	==, != (Equality)
	^ (Exclusive OR)
	&& (AND)
	 (OR)
	=, +=, -=, *=, /=, %= (Assignment operator)

Key Rule of Thumb

Logical operators have lower precedence than relational operators, and relational operators have lower precedence than arithmetic operators:

Arithmetic Operators → Relational Operators → Logical Operators

Associativity Patterns

1. Left-Associative Operators

All binary operators (operators that require two operands) except assignment operators process from left to right. Because addition (+) and subtraction (-) share the same precedence tier and are left-associative, they group as follows:

// Original Expression

`a - b + c - d`

// Equivalent Execution Paths

`((a - b) + c) - d`

2. Right-Associative Operators

Assignment operators are right-associative, meaning expressions group and evaluate from right to left.

// Original Expression

`a = b += c = 5`

// Equivalent Execution Paths

`a = (b += (c = 5))`

- **Execution Trace:** Suppose *a*, *b*, and *c* all start with an initial value of 1.
 - `c = 5` evaluates first. *c* is updated to 5, and the expression returns 5.
 - `b += 5` evaluates next. *b* becomes 1+5=6, and the expression returns 6.
 - `a = 6` evaluates last. *a* is assigned the value 6.



- **Final state:** a becomes 6, b becomes 6, and c becomes 5.

Note Java manages its evaluation sequence internally via structured operational rules. However, the final result generated by a Java expression evaluation will always match its standard arithmetic mathematical counterpart.

Debugging

Debugging is the systematic process of finding and fixing errors within a program. While syntax and runtime errors are typically surfaced directly by the compiler or Java interpreter, logic errors (commonly referred to as bugs) do not cause the program to crash, making them the most challenging errors to identify.

Common Debugging Methodologies

For short, straightforward programs, developers often rely on manual diagnostic techniques:

- **Hand-Tracing:** Manually reading through the source code line by line to verify data transitions and catch logic flaws.
- **Print Statements:** Inserting temporary output tracking lines (e.g., `System.out.println`) to print out active variable states and follow the program's real-time execution flow.

For large-scale or complex software applications, manually tracing execution becomes unmanageable. The most efficient approach is to use a dedicated debugger utility.

Debugger Utilities and Tools

The Java Development Kit (JDK) includes a native, command-line utility called `jdb`, which runs its own instance of the Java interpreter. Modern Integrated Development Environments (IDEs)—such as Eclipse, NetBeans, and IntelliJ IDEA—provide highly visual, integrated debuggers built directly into their workspaces.

Key Features of Modern Debuggers

- **Single-Statement Execution (Stepping):** Allows you to execute a single statement at a time, making it easy to see exactly how individual lines alter the application's memory and state.
- **Step Into vs. Step Over:**
 - **Step Into:** Enters a specific method call to execute its internal operations line by line.
 - **Step Over:** Executes a method in its entirety without entering its inner lines. This is ideal when you are confident a method works correctly or when dealing with built-in system instructions like `System.out.println`.
- **Setting Breakpoints:** Places markers on specific lines of code where the program will pause during execution. This allows you to jump directly to sections of code where you suspect a bug begins instead of stepping through the entire program from scratch.
- **Displaying Variables (Watch Windows):** Allows you to select and observe specific variables. The utility continuously updates and displays their values as you step through the code.
- **Displaying Call Stacks:** Provides a structural history of all active method calls, helping you visualize the execution path and overall architecture of your program.



GitHub: [Rashmi-Bansal35](#)

- **Modifying Variables:** Enables you to change a variable's value on the fly during a live debugging session. This lets you test how different data ranges behave without restarting the debugger or altering the source files.

Tip: Leveraging Debuggers to Learn Using an IDE debugger is not just for fixing broken code; it is one of the most effective ways to truly understand how Java handles memory allocations, conditional scoping, and structural logic behind the scenes. Maximize its use during your daily coding practice to catch mistakes early.